

Photobooth Renderer Handbook

Version und Zielgruppe

- Version: 2026-03-25
- Zielgruppe: Entwickler:innen / Techniker:innen, die den Capture-Flow, den Python Tool-Server und das Rendering betreiben oder erweitern

1. Architektur

Browser UI (JS/jQuery)

- `capture_bindings.js` startet den Flow
 - Konfig laden
 - Template-Slots ermitteln
 - Flow starten
- `capture_flow.js` führt aus:
 - Countdown
 - Liveview
 - Capture-Loop
 - Rendering
 - Finish-UI
- `capture_api.js` spricht `camerabridge.exe` per HTTP an:
 - Liveview
 - Capture
 - Settings

camerabridge.exe (HTTP API)

- liefert Liveview-Frames
- speichert Fotos als:
 - `Photo_1.jpg`
 - `Photo_2.jpg`
 - ...
- Request ohne Extension, Schreiben mit Extension

Python Tool-Server (`python_server.py`)

- stellt lokale HTTP-Endpunkte bereit:
 - `/ping`
 - `/render/collage`
 - Drucker
 - Service control
 - etc.
- schützt schreibende Endpunkte per API-Key
- schützt das Rendering per Lock
 - wichtig für Output-Index
 - wichtig für Copy-Operationen

Renderer-Modul (`render_core.py`)

- liest Template-XML:
 - Canvasgröße
 - Layers
 - optionale Greenwall-Flags
- lädt Photos und Assets
- verkleinert Photo-Layer vor der Weiterverarbeitung auf eine sinnvolle Arbeitsgröße
- optional Greenwall:
 - diff
 - chroma
 - auto
- baut und speichert die Collage
- kopiert Originale
- optional auch Greenwall-Kopien

Neu: `render.photo_work_scale` steuert die interne Arbeitsgröße für **alle** Photo-Layer.

- es wird **nur verkleinert**
- es wird **niemals hochskaliert**

2. Warum ein `session.json`-Snapshot sinnvoll ist

Ein Snapshot im Capture-Ordner ist sinnvoll für:

- **Determinismus**
 - Python rendert immer aus dem gespeicherten Snapshot
 - auch wenn JS abstürzt oder neu lädt
- **Debugging**
 - jede Session kann später reproduziert werden
- **Recovery**
 - bei einem Crash kann ein Worker den letzten Zustand sehen
- **Entkopplung**
 - JS muss nicht alles live an Python schicken
 - ein Pfad plus optionale Overrides reicht aus

Empfehlung:

- `session.json` zu Beginn erzeugen
- bei jedem Statuswechsel atomar im Capture-Hint-Ordner aktualisieren

3. Minimaler Status-Automat

In `session.json` reicht ein kleiner Status-Automat:

- INIT
- CAPTURING
 - mit `n_done / x_total`
- CAPTURE_DONE
- RENDERING
- DONE
- PRINTED
 - falls Print als eigener Status geführt wird
- ERROR
 - mit `code`
 - mit `message`

4. Empfohlenes `session.json`-Schema

```
{
  "id": "s1736760000000_abcd1234",
  "createdAt": "2026-03-25T10:15:00Z",
  "updatedAt": "2026-03-25T10:15:09Z",

  "status": "CAPTURING",
  "progress": { "done": 1, "total": 4 },

  "eventName": "Demo Event",
  "basePath": "D:\\Photos",
  "eventPath": "D:\\Photos\\EVENTS\\Demo Event",

  "photoTarget": 4,
  "expectedFiles": ["Photo_1", "Photo_2", "Photo_3", "Photo_4"],
  "photos": [
    { "slot": 1, "expectedName": "Photo_1", "file": "D:\\tmp\\Photo_1.jpg", "ts": 1736760000001 }
  ],

  "render": {
    "template": "D:\\booth\\templates\\active\\template.xml",
    "output_collage": "D:\\Photos\\EVENTS\\Demo Event\\final",
    "output_originals": "D:\\Photos\\EVENTS\\Demo Event\\original_copies",
```

```

    "prefix": "collage_",
    "ext": ".jpg",

    "render_config": null,
    "render_config_inline": {
        "render": {
            "resize_mode": "contain",
            "contain_bg": "#8c1212",
            "photo_work_scale": 1.5
        },
        "greenwall": {
            "enabled": true,
            "mode": "auto"
        }
    }
},

    "print": {
        "autoPrint": true,
        "printerCount": 1,
        "multiplePrint": true,
        "printCounter": 2
    },

    "error": null
}

```

Hinweise:

- `render.*` ist genau das, was `render_core.render_from_session()` bevorzugt ausliest
- `render.render_config_inline.render.photo_work_scale` wirkt auf **alle** Photo-Layer
 - auch wenn Greenwall deaktiviert ist
- Print-Daten werden **nicht** vom Renderer gebraucht
 - sie passen aber sinnvoll in den Snapshot für spätere Printer-Worker

5. Rendering-API: zwei saubere Wege

5.1 Rendering per HTTP

Empfohlen im Betrieb:

- POST `/render/collage`

Minimale Payload:

```

{
    "template": "booth/templates/active/template.xml",
    "input_dir": "booth/photos/original",
    "output_collage": "booth/photos/final",
    "output_originals": "booth/photos/original_copies",
    "prefix": "collage_",
    "ext": ".jpg",
    "render_config": null,
    "render_config_inline": {
        "render": {
            "resize_mode": "contain",
            "contain_bg": "#8c1212",
            "photo_work_scale": 1.5
        },
        "greenwall": {
            "enabled": true,
            "mode": "auto"
        }
    }
}

```

5.2 Rendering direkt als Python-Funktion

Nützlich für Tests oder CLI:

```
from render_core import render_collage_api

result = render_collage_api({
    "template": "D:/booth/templates/active/template.xml",
    "input_dir": "D:/booth/photos/original",
    "output_collage": "D:/booth/photos/final",
    "output_originals": "D:/booth/photos/original_copies",
    "prefix": "collage_",
    "ext": ".jpg",
    "render_config_inline": {
        "render": {
            "photo_work_scale": 1.5
        }
    }
})
print(result)
```

Ohne Server fehlen dir:

- API-Key-Schutz
- Locking gegen parallele Render-Requests
- einheitliche HTTP-Integration in JS

6. Render-Einstellungen

6.1 `render.resize_mode`

Steuert, wie ein Bild in den Zielbereich eingepasst wird:

- `stretch`
 - auf Zielgröße strecken
- `cover`
 - Zielbereich vollständig füllen
 - Beschnitt möglich
- `contain`
 - alles sichtbar
 - ggf. mit Balken oder Hintergrundfarbe

6.2 `render.contain_bg`

Hintergrundfarbe für `contain`, zum Beispiel:

- `"#RRGGBB"`
- `"#RRGGBBAA"`
- `"rgb(r,g,b)"`
- `"rgba(r,g,b,a)"`
- `[r,g,b]`
- `[r,g,b,a]`
- `"black"`
- `"white"`
- `"transparent"`

6.3 `render.photo_work_scale`

Steuert die **interne Arbeitsgröße** für alle **Photo-Layer** vor der weiteren Verarbeitung.

Eigenschaften:

- wirkt **auch ohne Greenwall**
- wirkt **vor**:
 - Greenwall
 - Rotation
 - Shadow

- Border
- finalem Platzieren
- verwendet die finale Layer-Content-Größe als Basis
- **nur Downscale**, niemals Upscale
- Standard: 1.5
- Alias: `render.photo_scale`

Praktische Bedeutung:

- kleine Collage-Slots werden intern deutlich kleiner verarbeitet
- große Vollbild-Layer bekommen mehr Arbeitsauflösung
- DSLR-Originale bleiben als Quelldatei erhalten
- der Renderer arbeitet mit einer passenden Zwischenauflösung schneller

Empfohlene Startwerte:

- 1.3 bis 1.5 für mehr Speed
- 1.5 als guter Standard
- 1.7 bis 2.0 nur, wenn feine Kanten oder hochwertige Greenwall-Ergebnisse wichtiger sind als Renderzeit

Beispiel:

```
{
  "render": {
    "resize_mode": "contain",
    "contain_bg": "#8c1212",
    "photo_work_scale": 1.5
  }
}
```

7. Greenwall

7.1 Aktivierung

Greenwall wird aktiv, wenn:

- im `template.xml` am Root-Tag `greenwall="1"` gesetzt ist
- und in `render_config.json` oder Inline-Config `greenwall.enabled=true` gesetzt ist

Legacy-Support:

- `greenwall.switch = "on" | "off" | "auto"`

Hinweis:

- wenn `greenwall="0"` und `greenwall.switch="auto"`, bleibt Greenwall aus
- `render.photo_work_scale` wirkt trotzdem weiter auf normale Photo-Layer

7.2 Modi

- **auto**
 - diff, wenn Referenzbild vorhanden
 - sonst chroma
- **diff**
 - Differenz-Key gegen Referenzbild
- **chroma**
 - klassischer Greenscreen anhand dominanter G-Komponente

7.3 Referenzbild (diff)

Gesucht in `input_dir` oder `input_dir/assets`:

- per XML `greenwall-src="..."`
 - Basename wird verwendet
- oder automatisch nach:
 - `greenwall.png`
 - `greenwall.jpg`
 - `greenwall.jpeg`

- case-insensitive

7.4 Hintergrundbild (optional)

Gesucht in `input_dir` oder `input_dir/assets`:

- per XML `greenwall-bg="..."`
 - relativer Pfad oder Basename
- oder automatisch nach:
 - `___greenwall.jpg`
 - `___greenwall.png`
 - `___greenwall.jpeg`
 - `greenwall_bg.*`
 - etc.

7.5 Debug-Masken

`greenwall.write_mask_debug=true` schreibt pro Foto eine Maske als PNG nach:

- `output_originals/original_greenwall/*_mask.png`

8. Output-Naming, Index und Parallelität

- der Output-Index wird aus dem Zielordner ermittelt:
 - `prefix + 000001 + ext`
- der Code ist **nicht parallel-sicher**, wenn mehrere Requests gleichzeitig rendern
- deshalb ist der Server-Lock `_RENDER_LOCK` korrekt und wichtig

9. JS-Integration

9.1 Snapshot schreiben

Empfohlen im Capture-Flow, sobald der Capture-Hint-Ordner bekannt ist:

- `session.json` anlegen
- bei jedem Slot und Statuswechsel atomar aktualisieren

9.2 Render starten

Variante 1:

- direkte Payload an `/render/collage` senden

Variante 2:

- nur `captureFolderHint` senden
- Python rendert aus `session.json`
- dafür ist ein Server-Endpunkt wie `/render/session` nötig

Wenn Render-Optionen aus dem Overlay übergeben werden, sollten mindestens diese Werte im Snapshot landen:

```
{
  "render": {
    "render_config_inline": {
      "render": {
        "resize_mode": "cover",
        "contain_bg": "#000000",
        "photo_work_scale": 1.5
      }
    }
  }
}
```

10. Troubleshooting

Template nicht gefunden

- `template`-Pfad in Payload prüfen
- `template`-Pfad in `session.render.template` prüfen
- bei Fallback-Suche sicherstellen, dass `booth_root` korrekt erkannt wird
 - `config/`-Verzeichnis vorhanden

Photos nicht gefunden

- `render_core` sucht `Photo_{index}.*` im `input_dir`
- sicherstellen, dass die Capture-Bridge dort wirklich speichert

Greenwall wirkt nicht

- prüfen, ob im Root von `template.xml` `greenwall="1"` gesetzt ist
- prüfen, ob `render_config_inline.greenwall.enabled=true` gesetzt ist
- prüfen, ob Referenzbild oder Hintergrundbild im richtigen Ordner liegt

Rendern ist langsam, obwohl Greenwall aus ist

- prüfen, ob `render.photo_work_scale` gesetzt ist
- typischer Startwert: 1.5
- `photo_work_scale` beschleunigt die interne Verarbeitung
- das Laden der Quelldatei selbst wird dadurch **nicht** beschleunigt
- zusätzlich prüfen:
 - Anzahl und Größe der DSLR-Dateien
 - Shadow oder Rotation bei vielen Layern
 - Zielordner mit sehr vielen Dateien
 - Kopieren der Originale nach `output_originals`

Bild wird unscharf

- `render.photo_work_scale` schrittweise erhöhen
 - z. B. von 1.5 auf 1.7
- zusätzlich prüfen, ob der Layer im Template sehr klein ist

Parallelität oder doppelte Indizes

- nur einen Render-Request gleichzeitig zulassen

11. Quick Reference

- Renderer-Funktion: `render_collage_api(payload, base_dir=None)`
- Snapshot-Render: `render_from_session(session_folder, base_dir=None)`
- Server-Endpunkt: POST `/render/collage`
- globaler Render-Fit: `render.resize_mode`
- Balkenfarbe für `contain`: `render.contain_bg`
- interne Arbeitsgröße für **alle** Photo-Layer: `render.photo_work_scale`
 - Standard: 1.5
 - nur Downscale
- Greenwall Master: `greenwall.enabled`
- Greenwall Legacy-Schalter: `greenwall.switch`